



Namespace, Увод в типовете дефинирани от потребителя

Namespace:

- пространство от имена
- инструмент за избягване на коллизии от имена

СИНТАКСИС:

```
namespace ns-name {  
    //  
}  
using namespace ns-name;  
using ns-name:: ...;
```

⚠ Пространствата от имена могат да предизвикат конфликт на имена.

```
namespace A {  
    void f();  
}  
namespace B {  
    void f();  
}  
f() -> КОНФЛИКТ
```

Енумерация -
тип, рестриктиран
до домейн от стойности,
които включват специално
дефинирани константи

○ $\forall$ енумератор съответства на целo число.

○ По default започват от 0.

○ Ако не е указано към кое число съм, то това е предният +1.

enum Y {

a = 11;

b; —> 12

c = 20;

d = 10;

e; —> 11

f; —> 12

g = e + f; —> 23

НЕ е ПРОБЛЕМ

Размер на променлива от тип enum:

- колкото е размерът на някой целочислен тип, в който могат да се запишат стойностите на енумераторите

В някои компилатори *
има изискване $\leq \text{sizeof}(\text{int})$.

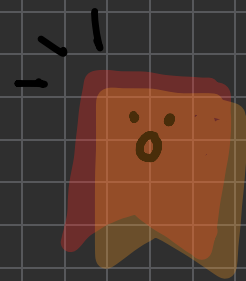
ПРОБЛЕМ 1: —> Enum е unscoped!

=> Поест енумераторите са нещо като ГЛОБАЛНИ променливи

```
enum Color {  
orange  
};
```

```
enum Fruit {  
orange  
};
```

=> не се компилира (дори orange да не е в друг enum, а в main)



ПРОБЛЕМ 2:

Неявно преобразуване от енумератор към число

```
enum color {  
red;  
};
```

```
enum Fruit {  
apple  
};
```

```
Color myColor = (color::red;  
Fruit myFruit = Fruit::apple;  
if(myColor == myFruit)
```

L> ще се компилира, а е нежелателно сравнение на 2 различни типа

Ползваме enum class:

scored enum

L> Няма неявно преобразуване от енумератор в число.

Решение:

=> При Проблем 1 ще се компилира.

=> При Проблем 2 няма да се компилира.

=> int x = color::orange НЕ става.

L> int x = (int) color::orange е ОК.

Структура:

- последователност от полета, които се запазват в опр. ред

структура - инициализация
клас - обект

I. Декларация:

```
struct Point {  
    int x;  
    int y;  
};
```

```
Point p;  
Point p{3,4};  
Point *ptr = new Point;
```

Ук-л към инициализация на Point



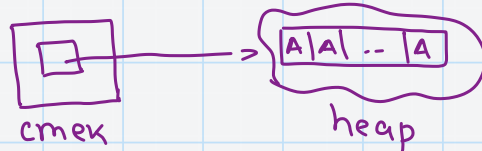
II. Построен

```
Point p;  
p.x;  
Point *ptr = new Point;  
(*ptr).x или ptr->x
```

III. Массиви от инициализация

```
A arr[10];  
↳ в стека
```

```
A * ptr = new A[n];
```



```
delete [] ptr;
```

IV. Влагане

3 НАЧИНА ДА ПОДАВАМЕ ВЪВ ФУНКЦИИ:

$f(\text{Point } p)$	$f(\text{Point } \&p)$	$f(\text{Point } *p)$
по копие	по резерв-я	по указател

⚠ Винаги, ако можем избягваме копирането!

⚠ Правим ли промени по обект/инициализация?
→ Ако не → по const подаване

ПОДАВАМЕ ВЪВ ФУНКЦИИ:



↳ Ако $f(\text{const Point } *p)$

↳ Можем да местим ук-я, но не и към какво сочи

VI. Размер и представяне в паметта

$\text{sizeof}(\text{MyStruct}) \geq \sum \text{sizeof}(\text{num})$
(num е член на myStruct)

$\text{sizeof}(\text{празна стр-ра}) = 1;$

Alignment requirement:

- разликата между два последователни адреса, на които може да съществуват променливи от този тип

$\text{sizeof}(\text{Union}) =$ размеро на най-големия променлива вътре

Примерни правила:

⚠ Големината на стр-та трябва да се дели на най-големия тип в нея.

⚠ Всяка елемент-данна да е на адрес, кратен на големината ѝ.

⚠ Промениливите са в реда, в който са декларирани в стр-та.

Endianness - Начин на подреждане на байтовете на типове данни

Big Endian

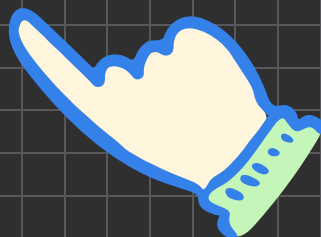
Най-старшият на 1-ва позиция

Little Endian

Най-младшият на 1-ва позиция

Union - последователност от полета, които споделят обща памет

Ако достъпим > 1 ел-т
 $\Rightarrow$ UNDEFINED BEHAVIOR



VII. Връщане от функции

struct A { }	A f() { A obj; return obj; }	✓ → връщане по <u>конец</u>
const A & f() { : return obj; }	const A* f() { : return obj; }	✗
A & f() { : return obj; }	A* f() { : return &obj; }	✗

// obj се унищожава, а връщаме & //

Увелишава нивото на obj, но не е правилно ⚠

A & f(A & ref) { : return ref; }	A* f() { A* ptr = new A; return ptr; }	✓
--	--	---

f1(A obj);	g(const x & x) {
f2(const A obj);	f1(x.obj); ✓
f3(A & obj);	f2(x.obj); ✓
f4(const A & obj);	f3(x.obj); ✓
f5(A* ptr);	f4(x.obj); ✓
f6(const A* ptr);	f5(x.obj); ✓
	}

// може само е по-късно

↕

1	2	3	4	5	6
1	2	3	4	5	6
1	2	3	4	5	6
1	2	3	4	5	6
1	2	3	4	5	6
1	2	3	4	5	6